

Linguagens Formais e Compiladores:

Aula 01 - Introdução

Profa. Jerusa Marchi

Universidade Federal de Santa Catarina
Departamento de Informática e Estatística

2024.1

Sumário

Teoria da Computação

- Conceitos Vistos

Linguagens Formais e Compiladores

- O que veremos

Linguagens de Programação - Definições Preliminares

- Compilador

- Interpretador

- Compilador/Interpretador (híbrido)

- Compilador X Interpretador

- Java como exemplo

Outros Programas Necessários Além do Compilador

Contexto de um compilador

- Primeira Abordagem

- Segunda Abordagem

Implementação de um Compilador

- O que é um “passo”?

- Impacto do Número de Passos

Teoria da Computação

Conceitos Vistos

- Estudo de máquinas e linguagens:
 - Computável
 - Decidível
 - Tratável
- Tese de Church-Turing
- Algoritmos $\times$ Procedimentos

Teoria da Computação

Conceitos Vistos

- Classes de Máquinas (AF, AP, MT);
- Classes de Linguagens:
 - Regulares
 - Livres de Contexto
 - Sensíveis ao Contexto
 - Recursivas
 - Recursivamente Enumeráveis
- Representação Finita de Linguagens possivelmente infinitas.

Linguagens Formais e Compiladores

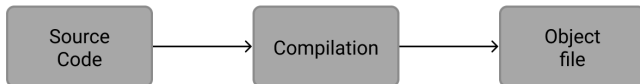
O que veremos

- Estudar classes de linguagens, seus mecanismos de representação e suas propriedades:
 - Gramáticas
 - Expressões Regulares
 - Máquinas
- Aplicações práticas dos formalismos e propriedades no processo de desenvolvimento e compilação de linguagens.

Linguagens de Programação

Definições Preliminares: Compilador

- Todo e qualquer código fonte precisa ser **traduzido** para que possa ser executado pela máquina;
- Um **Compilador** é um programa que recebe como entrada um programa em uma linguagem de programação fonte e o traduz para uma linguagem objeto equivalente.
 - Um papel importante do compilador é reportar erros no programa fonte durante o processo de tradução:

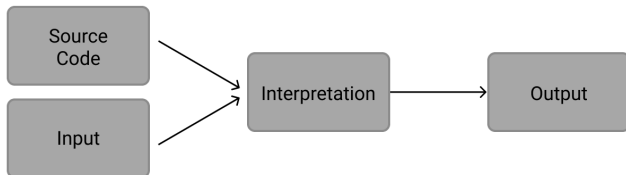


- Se o programa objeto for um programa em uma linguagem de máquina executável, este poderá então ser chamado pelo usuário.

Linguagens de Programação

Definições Preliminares: Interpretador

- Um **Interpretador** é outro tipo comum de processador de linguagem, porém...

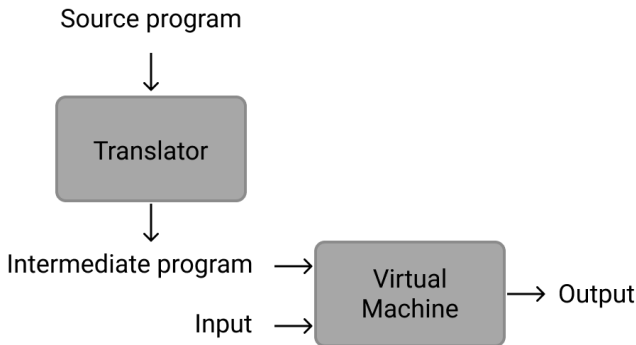


- Ao invés de produzir um programa objeto, um interpretador executa diretamente operações especificadas no programa fonte sobre as entradas fornecidas pelo usuário;
- Cada linha é interpretada. Se correta, é executada.

Linguagens de Programação

Definições Preliminares: Compilador/Interpretador (híbrido)

- O programa fonte é lido e traduzido em um código intermediário. Ao mesmo tempo, é inicializada uma máquina virtual que recebe este código como entrada e o executa, retornando o resultado.



Linguagens de Programação

Definições Preliminares: Compilador X Interpretador

	Compilador	Interpretador
Tempo de compilação	Grande	-
Tempo de execução	+ Rápido	+ Lento
Consumo de memória	Maior	Menor
Código fonte	Oculto	Visível
Otimização	Sim	Não
Depuração	+ Complexa	+ Simples
Execução com erro	Não	Sim

Linguagens de Programação

Definições Preliminares: Java como exemplo

- Os processadores da linguagem Java são híbridos - combinam compilação e interpretação;
- O programa fonte é compilado, gerando uma forma intermediária - Java bytecode;
- O bytecode é interpretado por uma Java Virtual Machine (JVM):
 - A máquina pode diferir daquela em que o bytecode foi gerado.

Outros Programas Necessários Além do Compilador

Além do compilador, outros programas podem ser necessários para a criação do programa objeto executável:

Pré-processador:

O código fonte pode estar dividido em módulos armazenados em arquivos separados ou conter macros que precisam ser expandidas.

Assembler / Montador:

O compilador pode gerar uma linguagem simbólica (Assembly) que precisa ser traduzida para código de máquina realocável.

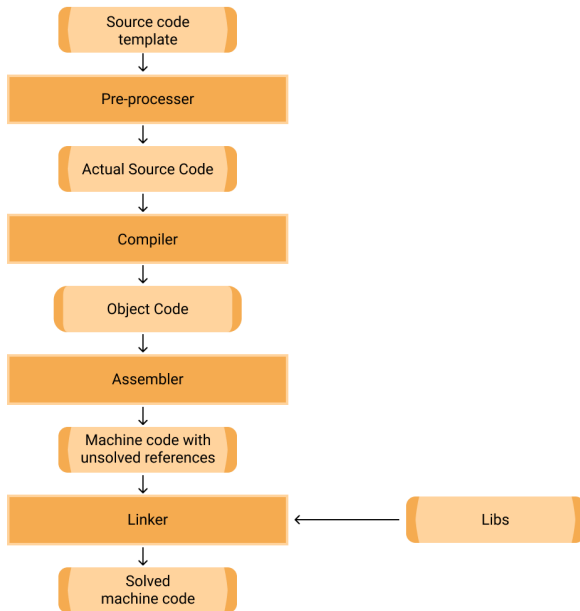
Link Editor / Linker / Editor de Ligação:

Programas grandes são compilados em partes e seus códigos de máquina realocáveis precisam ser unidos. Além disso, ainda há a necessidade de linkar o código com bibliotecas, resolvendo endereçamentos de memória.

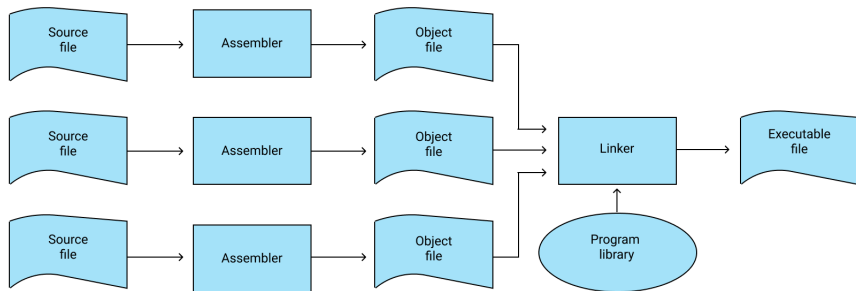
Loader / Carregador:

Os arquivos objeto executáveis são carregados para a memória.

Contexto de um compilador



Contexto de um compilador



Estrutura Geral do Compilador

Primeira Abordagem

- **Análise:**

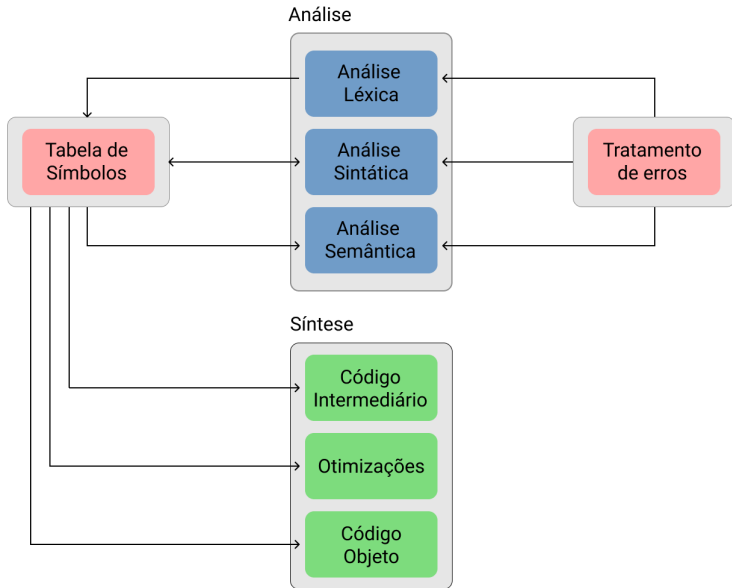
- Divide o programa fonte nas partes que o constituem e cria uma representação intermediária.
 - (1) Léxica;
 - (2) Sintática;
 - (3) Semântica.

- **Síntese:**

- Constrói o programa alvo a partir da representação intermediária.
 - (1) Geração de código intermediário;
 - (2) Otimização de código;
 - (3) Geração de código-objeto.

Estrutura Geral do Compilador

Primeira abordagem



Estrutura Geral do Compilador

Segunda abordagem

- **Interface de Vanguarda (front-end):**

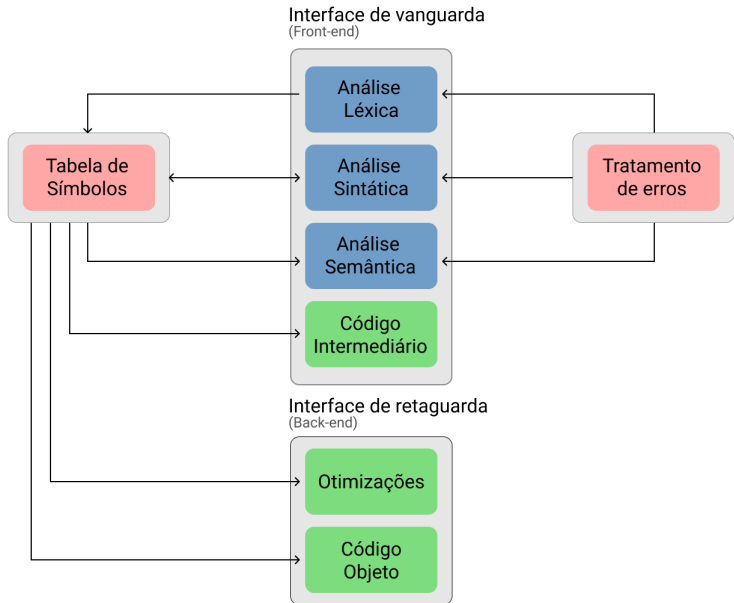
- Dependem da linguagem, independe da máquina alvo.
 - (1) Análise Léxica;
 - (2) Análise Sintática;
 - (3) Análise Semântica;
 - (4) Geração de código intermediário.

- **Interface de Retaguarda (back-end):**

- Constrói o programa alvo a partir da representação intermediária.
 - (1) Otimização de código (opcional);
 - (2) Geração de código-objeto.

Estrutura Geral do Compilador

Segunda abordagem



Implementação de um Compilador:

O que é um “passo”?

Passo: Passagem completa do programa compilador sobre o programa fonte que está sendo compilado. Gera um arquivo de saída.

- Cada fase pode ser implementada em um passo separado, de modo a gerar resultados parciais como saída.

Implementação de um Compilador:

O que é um “passo”?

Também pode ser centrado na análise sintática (compilador de 1 único passo):

- O analisador sintático solicita ao analisador léxico que lhe forneça os tokens;
- Ao receber uma estrutura sintática completa, com esta correta, o analisador sintático chama o analisador semântico;
- Estando a estrutura correta semanticamente, o analisador sintático solicita a geração de código intermediário;
- Traduz-se a estrutura e o controle volta para o analisador sintático.

Implementação de um Compilador

Impacto do Número de Passos

- Quanto maior o número de passos:
 - ↑ Maior complexidade do compilador;
 - ↑ Maior tempo de compilação;
 - ↑ Maior a possibilidade de executar otimizações; e
 - ↓ Menor consumo de memória.